Assignment 4
CS 3443 Application Programming

Tic-tac-toe

Tic-tac-toe is a game for two players. Players take turns playing their marks (X and O) on an initially empty 3-by-3 grid of squares. On each turn, a player puts his/her mark in an unmarked square. A player wins a game by being the first to have his/her marks occupy all of a column, row, or diagonal. If the grid is filled with neither player winning, then the game is a draw.

It is common for two players to play a series of games keeping track of the number of each player's wins as well as the number of draws. To assist such players, a JavaFX application can be created for playing tic-tac-toe. The application's user interface uses Labels and Buttons to communicate with the players. See Figure 1. The elements are aligned vertically in the following order:

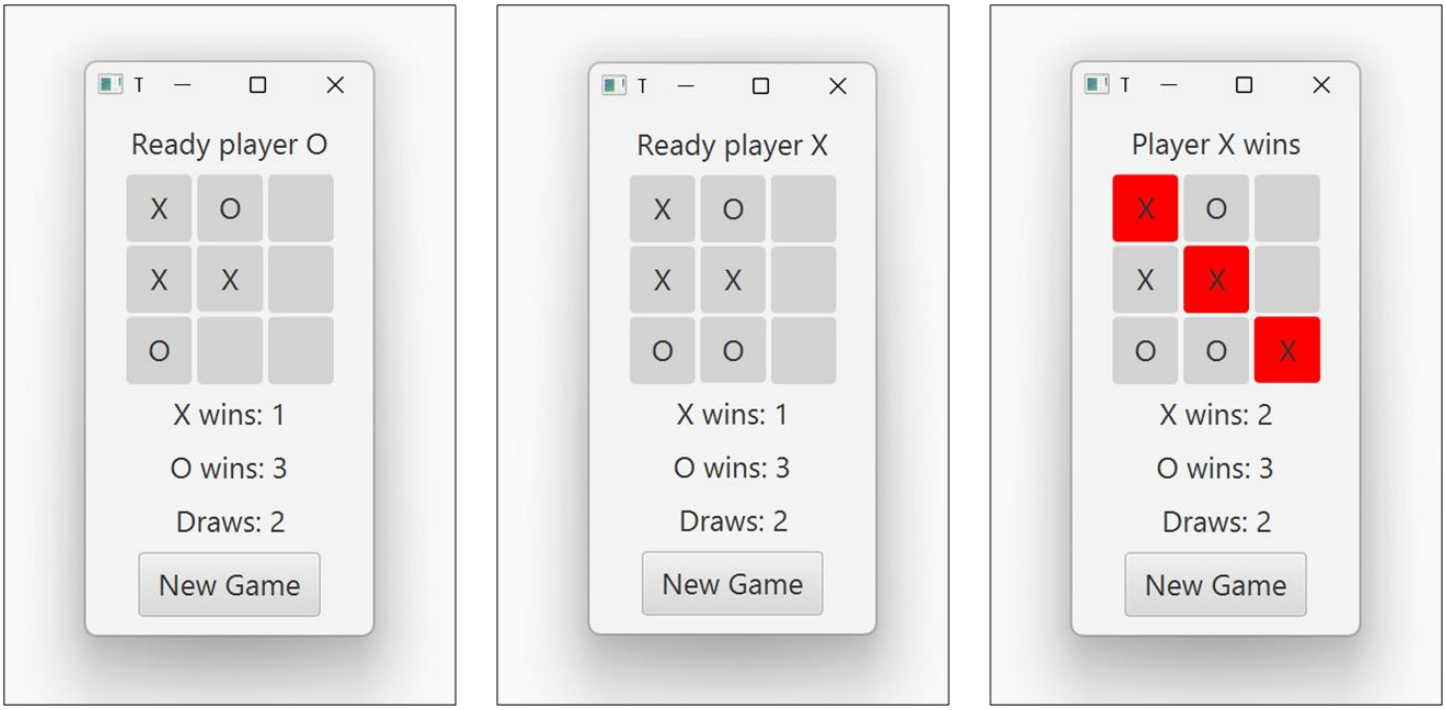


Figure 1. Two consecutive moves on the tic-tac-toe application

1. A label providing directions to the players. Its possible messages are: "Ready Player X", "Ready Player O", "Player X wins", "Player O wins", and "The game is a draw". The status is updated after each player's turn.

2. A 3-by-3 arrangement of buttons forming a playing board. Players click on buttons to place their markers which then appear on the buttons. If a player clicks on a button that already has a marker, nothing happens, and the player can click on another button. A game ends when one player has won or after 9 turns. After a game ends, additional clicking on buttons has no effect. If a player wins, then the color of the buttons of the player's winning row, column or diagonal are changed from to red. If there is more than one winning row, column or diagonal, only one of them is changed to red.

3. A label displaying the number of games player X has won. This is updated immediately after player X wins a game.

4. A label displaying the number of games player O has won. This is updated immediately after player O wins a game.

5. A label displaying the number of games that are draws. This is updated immediately after a game is finished if no player wins.

6. A new game button. When this button is clicked, a new game is immediately started even if the current game has not yet ended. The 9 buttons forming the board are cleared of their markers and all buttons are reset to light gray. Players alternate

first-turns, so the first player of the next game is the player who went second in the previous game. If a game is not completed when the new game button is clicked, then that game counts as neither a win nor a draw.

Programming Tic-tac-toe

Create a JavaFX project named "a4_abc123" where abc123 is your UTSA ID. The project should use Java version SE 21.

Use the attached Tictactoe2.fxml file. Open it in SceneBuilder to examine the fx:ids of the labels and buttons and to examine the On Action property of the buttons. Do not change the Tictactoe2.fxml file.

Use the Model-View-Controller pattern and implement the following packages and classes. The classes mentioned below in the model package are in a UML class diagram saved in the Violet file tictactoe.class.violet.html. Use the Violet tool to create a class diagram including all the classes you defined for your program.

The `application.model` package is for code that implements the Tic-tac-toe game. It must contain at least the following (you may define additional classes and methods):

- The `Marker` enum has just two values: X and O. It is used for marking the game board. Use the following as your definition of Marker.java:

```
package application.model;
public enum Marker { X, O }
```

- The `Position` class is used for representing the row and column of a location on the playing board. It has a constructor and two getters: `getRowNum` and `getColNum`.
- The `TicTacToeBoard` class represents the playing board as a 3-by-3 array of `Markers`. Note that Java enum values include null, so a newly allocated array of `Markers` has all null values. Along with a zero-argument constructor, it has the following methods. The `TicTacToeBoard` class implements the rules of the game, keeps track of the current player and the numbers of draws and players' wins.
    - The `putMarker` method takes a Position object as its only argument and sets the value at the given position on the playing board to the current player's `Marker` if that location has not yet been used in the current game. This method is also responsible for completely processing a player's move and updating the application view either directly or by calling helper methods. The color of a button that is part of a winning row, column or diagonal should be changed to "red". Make sure that unacceptable actions such as clicking on a button that is already marked or clicking on any button in the array after a game is ended are properly handled.
    - The `winningPositions` method takes a `Marker` as an argument and determines if the player using that `Marker` has won the game. If the player has won the game, then the method returns a 3-element array of the `Positions` of the player's winning row, column, or diagonal. If the player has not won the game as reflected by the current state of the playing board, then `null` is returned. If there is more than one row, column, or diagonal that is a winning position, then the method can return an array of positions for any one of them.
    - The `clearBoard` method sets each location on the playing board to `null` and updates the application view. The color of buttons in the application view should be "lightgray".

The `application.view` package is for code used to represent the application view. This package has a class named `MainView`. Use a 3-by-3 array to store the tic-tac-toe board buttons. The `MainView` class has the usual setters for the labels. For the buttons that form the tic-tac-toe board, there is a single `setButton` method that takes as arguments: a button, a row number, and a column number and sets the button at that position in the tic-tac-toe board. The methods whose names have the prefix "`display`" are for changing the text in a label or on a button. The `changeBackgroundColor` method changes the color of a button.

The `application.controller` package has the application controller code. This package has a class named `MainController`. This class has an `initialize` method that takes no arguments and is called by JavaFX after the @FXML variables have been given values through injection. The `initialize` method creates the `TicTacToeBoard`

and `MainView` instances, passes the labels and buttons to the `MainView` instance, and gives the `MainView` instance to the `TicTacToeBoard` instance, The `MainController` class also has the `newGameButtonClicked` and `gridButtonClicked` methods which are callled by JavaFX when the user clicks on the New Game button or clicks on a tic-tac-toe board button.

DRY Principle

Note that it would be a violation of the DRY principle if the application controller had nine different methods for each of the nine buttons forming the tic-tac-toe board. Each method would call the same method: `putMarker`, but pass a different Position object.

To comply with the DRY principle, there is a single controller method, `gridButtonClicked`, for the nine board buttons. This means that during initialization, the application controller should assign to each button its position on the tic-tac-toe board represented as a `Position` object. Use each button's `setUserData` method to do this. When the `gridButtonClicked` method is called, it should retrieve the button from its event argument and call `getUserData` to retrieve the button's `Position` object.

UML Class Diagram

Complete the given UML class diagram in Violet. Include all of the classes and methods that you defined. Do not include the `Main` class. Export your diagram to an image file named tictactoe_abc123 where abc123 is your UTSA student ID.

Programming Style

Use good programming style. Your comments should include a description of each class and of each method in Javadoc. Your implementation should follow the DRY principle as described above.

Testing

First create the application model without using the `MainView` class by having it write to the console to describe what it is doing. After testing this version of the model, create the `MainView` class and change the model to use it. This will require a simplified version of the application controller. After the `MainView` class has been tested, complete the application controller and test it to make sure that the model is correctly used when the user clicks on a button.

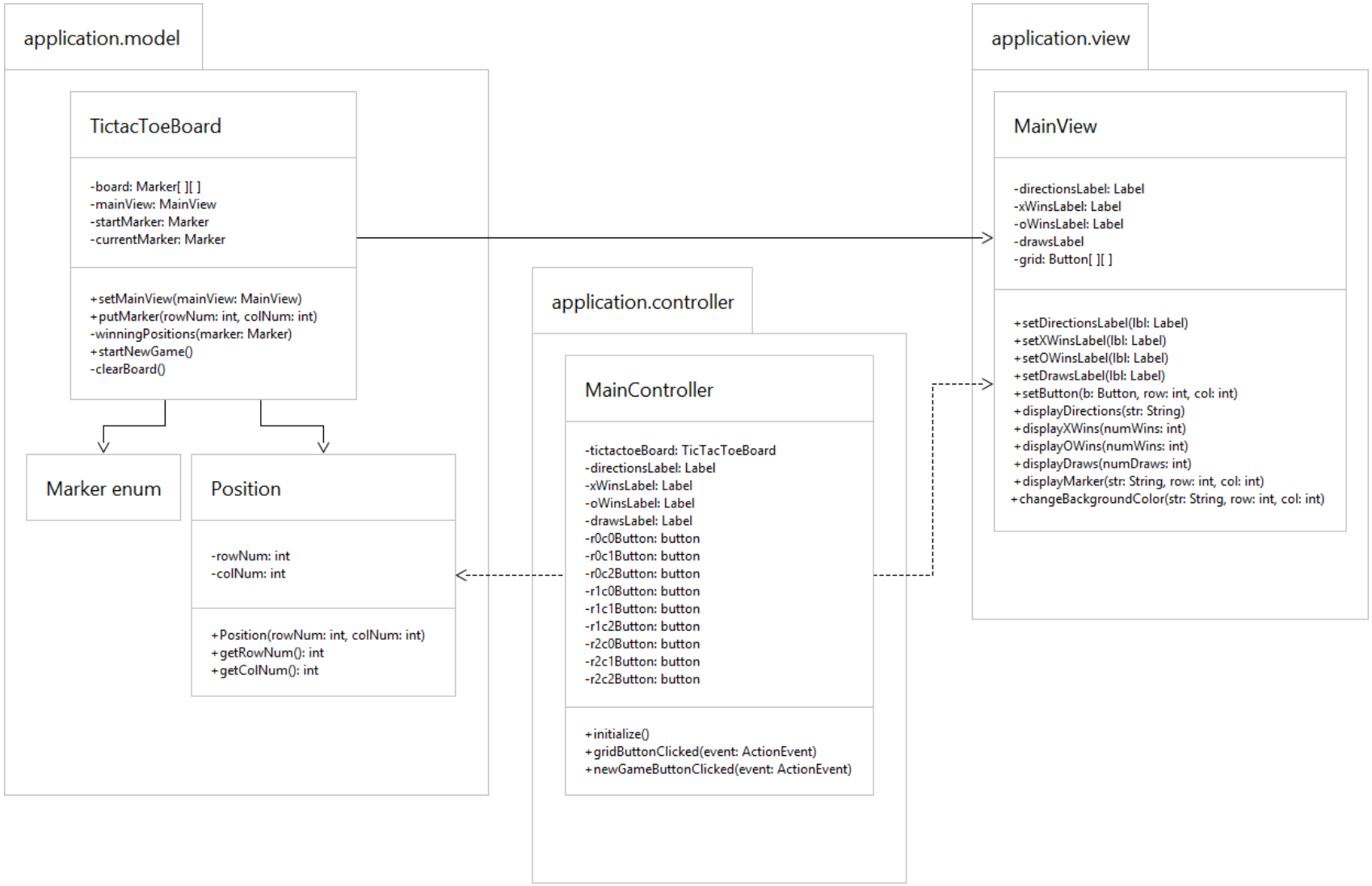
Deliverables

- Create a JavaFX project named a4_abc123 where abc123 is your UTSA ID
- Implement the classes as described. You may define additional methods in the classes.
- Export your project to a zip file named a4_abc123.zip where abc123 is your UTSA student ID.
- Verify that the zip file contains your work.
- Submit your zip file and class diagram image file on Canvas
- Verify that you have submitted your zip file correctly.

Rubric

1.	Good programming style: Javadoc comments, indentation, names, DRY	10%
2.	Correctly created Model	25%
3.	Correctly created View	25%
4.	Correctly created Controller	25%
5.	UML class diagram	15%
	Total	100%

No credit is given for submissions that do not compile.


application.model
TictacToeBoard
-board: Marker[ ][ ]
-mainView: MainView
-startMarker: Marker
-currentMarker: Marker
+setMainView(mainView: MainView)
+putMarker(rowNum: int, colNum: int)
-winningPositions(marker: Marker)
+startNewGame()
-clearBoard()
Marker enum
Position
-rowNum: int
-colNum: int
+Position(rowNum: int, colNum: int)
+getRowNum(): int
+getColNum(): int
application.controller
MainController
-tictactoeBoard: TicTacToeBoard
-directionsLabel: Label
-xWinsLabel: Label
-oWinsLabel: Label
-drawsLabel: Label
-r0c0Button: button
-r0c1Button: button
-r0c2Button: button
-r1c0Button: button
-r1c1Button: button
-r1c2Button: button
-r2c0Button: button
-r2c1Button: button
-r2c2Button: button
+initialize()
+gridButtonClicked(event: ActionEvent)
+newGameButtonClicked(event: ActionEvent)
application.view
MainView
-directionsLabel: Label
-xWinsLabel: Label
-oWinsLabel: Label
-drawsLabel
-grid: Button[ ][ ]
+setDirectionsLabel(lbl: Label)
+setXWinsLabel(lbl: Label)
+setOWinsLabel(lbl: Label)
+setDrawsLabel(lbl: Label)
+setButton(b: Button, row: int, col: int)
+displayDirections(str: String)
+displayXWins(numWins: int)
+displayOWins(numWins: int)
+displayDraws(numDraws: int)
+displayMarker(str: String, row: int, col: int)
+changeBackgroundColor(str: String, row: int, col: int)